SMART WATER APP STUDENT GUIDE

Guide 01 | Set up | Clone | Run | Develop with AI

Use this guide to run Smart Drops on your computer, keep a working Git copy, and make your first change in an IDE. The supplied app uses Node.js and Vite with HTML, CSS and JavaScript. Its building model, simulated data, graphics and guides are included.

1 PREPARE	2 CLONE	3 RUN	4 DEVELOP	5 CHECK
Install Node and an IDE	Get your working copy	Start npm run dev	Edit with AI assistance	Test and save in Git

1. Prepare your computer

- Install Node.js 22.12 or newer from https://nodejs.org/. npm is included. Restart your editor after installation.
- Install Git from https://git-scm.com/downloads and an editor such as Visual Studio Code from https://code.visualstudio.com/ or Cursor. Keep the project in a permanent local folder, for example C:\EDU\SmartWaterApp.
- Use GitHub Copilot, Codex or another coding assistant if available to you. An AI account or plan may be required for those tools; the water app itself has no sign-in requirement.

Starter repository: https://github.com/smart-water-EDU/SmartWaterApp. Viewing or cloning a public repository does not require a GitHub account. Pushing work requires permission or your own repository.

2. Clone and open the project

Option A - Use your editor

In VS Code, press Ctrl+Shift+P and choose Git: Clone. Paste the HTTPS repository address below, choose the parent folder and select Open when cloning finishes. In Cursor, use its clone/open-project workflow for the same repository.

```
https://github.com/smart-water-EDU/SmartWaterApp.git
```

Option B - Use a terminal

Open Git Bash or a terminal in your chosen parent folder and run:

```
git clone https://github.com/smart-water-EDU/SmartWaterApp.git
cd SmartWaterApp
```

Open this folder in your IDE. It must contain package.json and index.html. Clone once; reopen the same folder for later sessions. Your local folder may have a different name, such as smart-water-app.

3. Install once and start locally

Open Terminal > New Terminal inside the project. Check the tools, install dependencies and start Vite:

```
node --version
npm --version
npm install
npm run dev
```

Open http://127.0.0.1:5173/ if the browser does not open automatically. This is an address on your own computer. Keep the Vite terminal running; save an edited file to see the change. Ctrl+C in that terminal stops Vite. Restart with npm run dev next time; install again only when dependencies change.

Do not double-click index.html to run this version. Node runs the development tools; index.html remains the browser entry point. Internet is needed for installation, Git/AI services and the optional weather API. The supplied simulation and saved designs use local data.

4. Choose your AI development environment

- VS Code with Copilot: enable AI features or open Chat, sign in if requested, and give the assistant access to this project. Use the editor's Source Control view to inspect every changed file.
- Cursor: open the same local folder and use its AI chat/agent. Run npm commands in the project terminal. Review the proposed changes before keeping them.
- Codex: open or select the local project in the Codex app or a supported IDE integration. Start by asking it to explain a file; review its changes and tests before continuing. See the official setup link at the end.
- Lovable is a separate browser-based workflow. Its Upload code workflow can import a repository ZIP into a new project; Git sync then creates a new repository. It does not attach to this existing repository through Git sync. Keep the local starter copy and agree the import/submission route with your teacher before using it.

5. Explore and make your first change

Start with Monitoring, Management, Maintenance and Development. Change the simulation clock, select a floor/module, inspect sensor gauges, and open Development > Additional Systems. The heat and rainwater designers already work; red STUDENTS EXTENSIONS panels identify places for new ideas.

Create a local branch before editing:

```
git switch -c student/my-idea
```

Begin with a small styling change, then use Guide 02 for an engineering extension. Suggested prompts:

READ ONLY: Read README.md and explain src/app.js, src/components/, src/styles/ and data/. Identify the STUDENT EXTENSION markers. Do not modify files.

FIRST EDIT: Make the selected tab slightly clearer. Change only the relevant CSS. Preserve the layout, data and calculations. Explain the selectors changed and how I can check the result.

Review the diff, try the feature, then run the checks in a second project terminal while Vite stays open:

```
npm test
npm run build
```

The build creates dist/ for deployment. Source changes belong in src/, index.html, data/ or docs/, not in dist/ or node_modules/. Build/start validate the water inputs and regenerate derived sensors; they do not intentionally regenerate the supplied consumption baseline.

6. Find the right files and save your work

- src/app.js connects tabs and controls. src/components/ holds the designers, charts and network interaction. src/styles/ holds appearance; the building SVG is embedded in index.html.
- src/heat-recovery-model.js, src/rainwater-model.js and src/pump-model.js separate engineering calculations from their interfaces. tests/ contains focused checks.
- data/ holds building/profile JSON, separate cold/hot module CSVs and all sensor channels for 8,760 hours of 2026. Run npm run data:build after intentional source CSV edits. npm run data:generate deliberately replaces the supplied water baseline; do not use it for a proposed design.
- Designs and scenarios are saved in your browser for this origin. Git saves source files; it does not automatically include browser-only designs. Keep the same address/browser and document or export scenario inputs for submission.

Use git status and git diff to review changes. Commit selected files with a clear message. Keep work on your own branch/fork or the teacher's assigned repository; do not push to the shared main branch without permission. Before updating from the starter repository, commit or back up your work and follow the teacher's merge instructions. Ask for help with conflicts; avoid force-push or broad file deletion.

7. Common problems and official help

- npm cannot find package.json: the terminal is in the wrong folder. Open the folder containing package.json, then start a terminal there.
- The local address will not open: check that npm run dev is still running. Port 5173 in use means another copy may already be running; inspect that terminal before closing it.
- PowerShell blocks npm.ps1: use npm.cmd install and npm.cmd run dev, or choose the Git Bash terminal profile. An old Node version requires upgrading Node and reopening the terminal.